

Monitor

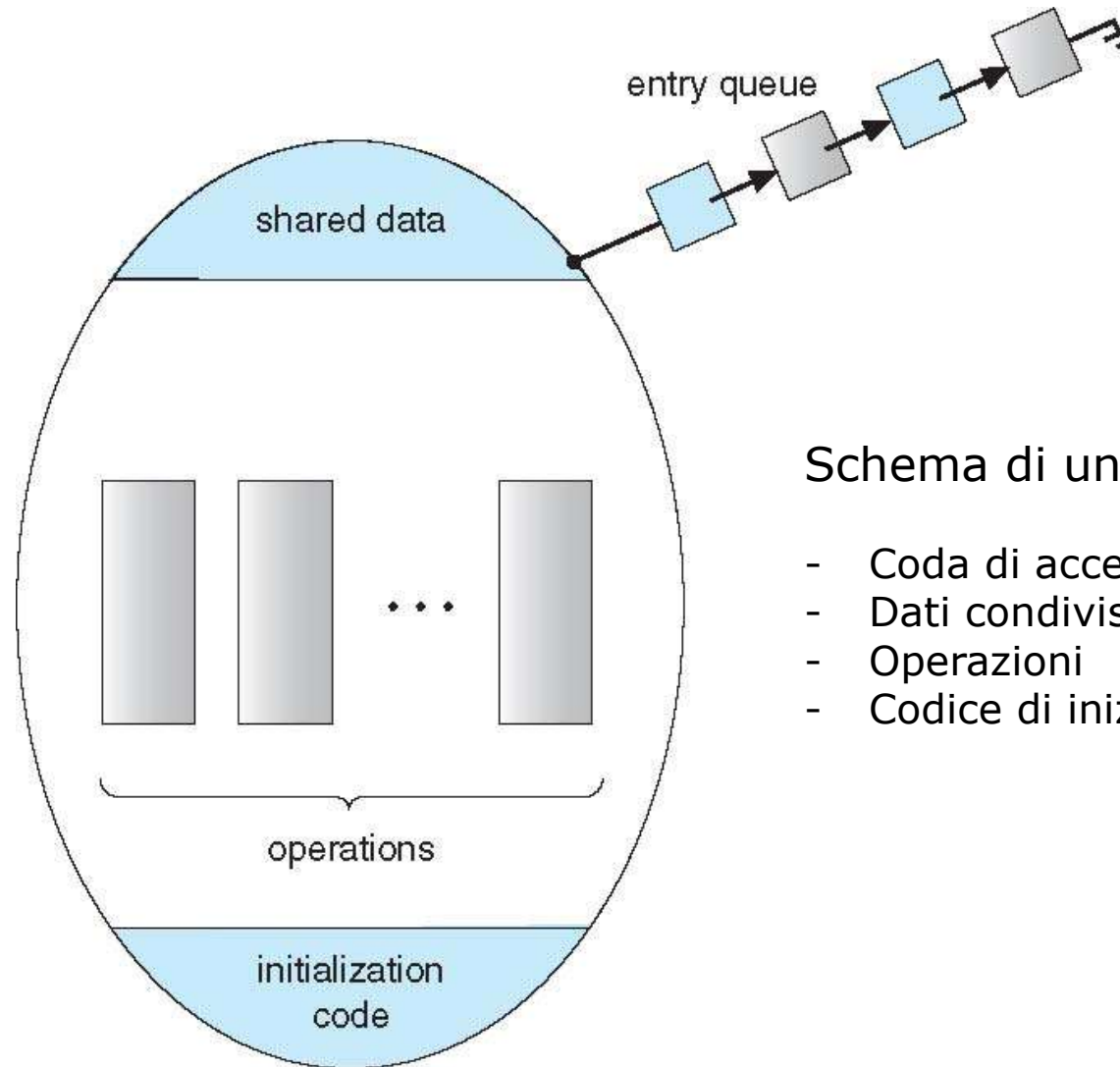
- Astrazione di alto livello che fornisce un meccanismo di sincronizzazione (introdotti nel 1974 da Hoare)
- *Tipo di dato astratto*, variabili condivise accessibili con procedure predefinite
- Solo un processo alla volta può essere attivo nel monitor
- Dati privati
- ... ma non potente abbastanza da modellare alcuni schemi di sincronizzazione
 - Introdotte le variabili di condizione

```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { ... }

    procedure Pn (...) {.....}

    Initialization code (...) { ... }
}
}
```

Rappresentazione di un Monitor



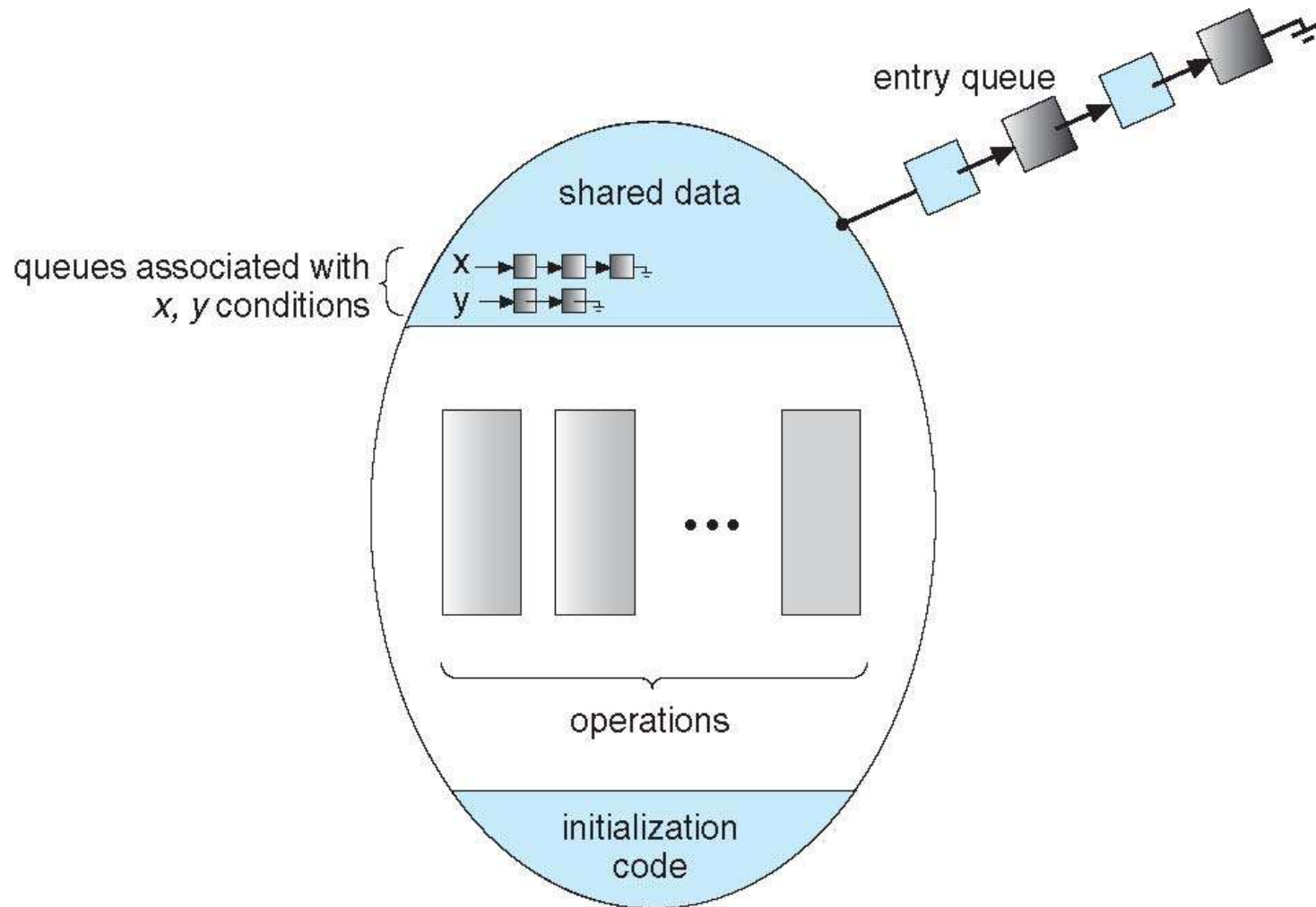
Schema di un Monitor:

- Coda di accesso
- Dati condivisi
- Operazioni
- Codice di inizializzazione

Variabili di Condizione

- `condition x, y;`
- Due operazioni sono permesse su una variabile di condizione:
 - `x.wait()` – il processo è sospeso finché non avviene `x.signal()`
 - `x.signal()` – riprende uno dei processi (se c'è) che ha invocato `x.wait()`
 - ▶ Se non c'è `x.wait()` sulla variabile, non c'è alcun effetto

Monitor con Variabili di Condizione



Scelte su Condition Variables

- Se il processo P invoca **`x.signal()`** e il processo Q è sospeso in **`x.wait()`** che succede?
 - Q e P non possono eseguire in parallelo nel monitor.
 - ▶ Se Q viene ripreso, P deve attendere
- Tra le opzioni abbiamo:
 - **Signal and wait** – P aspetta finché Q o lascia il monitor o aspetta un'altra condizione
 - **Signal and continue** – Q aspetta finché P lascia il monitor o aspetta un'altra condizione
 - Entrambe hanno pros e cons – implementatore del linguaggio decide
 - ▶ La seconda lascia al processo in esecuzione, ma la variabile attesa potrebbe cambiare
 - Monitor implementato in Concurrent Pascal fa compromesso
 - ▶ P esegue signal e subito lascia il monitor, quindi Q è ripreso
 - ▶ Implementato in altri linguaggi (Mesa, C#, Java)

Monitor Implementati con Semafori

□ Variabili

```
semaphore mutex; // (initially = 1)  next usato dal
semaphore next;  // (initially = 0)  processo segnalante
int next_count = 0;                  per sospendersi
```

□ Ogni procedura *F* sostituita da

Signal and Wait

```
wait(mutex) ;
```

```
...
```

```
body of F;
```

```
...
```

```
if (next_count > 0)
```

```
    signal(next)
```

```
else
```

```
    signal(mutex) ;
```

Accesso esclusivo

Se altri in attesa
lascia, altrimenti
sblocca

□ La mutua esclusione nel monitor è assicurata

□ Implementazione con **signal-and-wait**

Monitor Implementation – Condition Variables

- Per ogni condition var **x**, abbiamo:

```
semaphore x_sem; // (initially = 0)
int x_count = 0;
```

- La **x.wait** implementata come:

```
x_count++;
if (next_count > 0)
    signal(next);
else
    signal(mutex);
wait(x_sem);
x_count--;
```

Se prossimo in attesa
passa a lui, altrimenti
rilascia il mutex

Aspetta su x
Esci abbassando il cont

Monitor Implementation

□ La `x.signal` implementata come:

```
if (x_count > 0) {  
    next_count++;  
    signal(x_sem);  
    wait(next);  
    next_count--;  
}
```

Se prossimo in attesa, lascia
a lui e rimettiti in coda di
attesa per il monitor

sblocca la var x

Attendi su next
Quando entri aggiorna next

Ripresa dei Processi nel Monitor

- Se molti processi accodati sulla condizione x e avviene un x.signal() quali riprendere?
- FCFS spesso non adeguato
- **conditional-wait** come x.wait(c)
 - Dove c è il **priority number**
 - Processo con numero più basso (highest priority) è il prossimo schedulato (es. tempo di esecuzione più basso)

Single Resource allocation

- Allocata una singola risorsa tra processi in competizione usando i numeri di priorità che specificano il tempo massimo di utilizzo della risorsa

```
R.acquire(t) ;  
    ...  
    access the resurce;  
    ...  
  
R.release;
```

- Con R istanza di tipo **ResourceAllocator**

Monitor per Singola Risorsa

```
monitor ResourceAllocator
{
    boolean busy;
    condition x;
    void acquire(int time) {
        if (busy)
            x.wait(time);
        busy = TRUE;
    }
    void release() {
        busy = FALSE;
        x.signal();
    }
    initialization code() {
        busy = FALSE;
    }
}
```

Single Resource allocation

- Allocated a single resource among processes in competition using the priority numbers that specify the maximum time of use of the resource

```
R.acquire(t);  
...  
access the resource;  
...  
  
R.release;
```

- With R instance of type **ResourceAllocator**
- But the monitor does not guarantee correct execution
 - Access to resources without having the permissions, double request, release of resources without use, non release
 - Control of access defined in the monitor
 - But it must be controlled that all use it correctly

Liveness

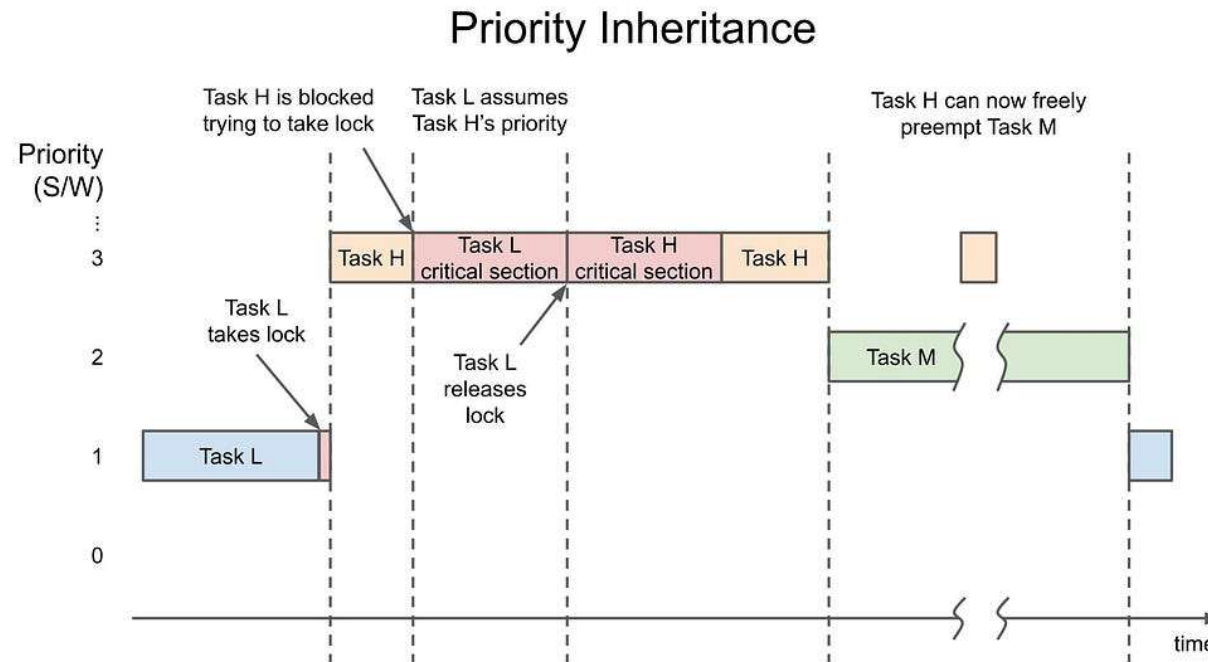
- Un processo che prova ad entrare in sezione critica potrebbe attendere indefinitivamente
- Violato **Progresso** e **Attesa Limitata**
- **Liveness:**
 - insieme di proprietà che un sistema deve soddisfare per garantire che i processi facciano progressi durante il ciclo di vita della loro esecuzione.
 - Un processo che attende indefinitamente è un esempio di “mancanza di liveness” (liveness failure).
- Esempi di situazioni che possono portare a liveness failure:
 - Ciclo infinito
 - Attesa indefinita
 - Deadlock
 - Inversione di priorità

Inversione di Priorità

- Processo ad alta priorità che deve accedere a risorse del kernel tenute da un processo a priorità più bassa
- Più complicato se il processo è prelazonato da un altro ancora
- Tre processo $L < M < H$
 - H chiede il semaforo S tenuto da L
 - H deve aspettare che L liberi il semaforo
 - ... M diventa runnable e prelazona L
 - Il processo M influenza l'attesa di un processo H (a più alta priorità) sul semaforo S tenuta da L
- Priority-inheritance protocol
 - Quando processi accedono ad una risorsa richiesta da un processo ad alta priorità, ereditano la priorità del processo. Poi tornano ai valori originali
 - L avrebbe ereditato la priorità di H bloccando M (rientra con priorità L al rilascio), la competizione tra H ed M è così vinta da H

Inversione di Priorità

- Processo ad alta priorità che deve accedere a risorse del kernel tenute da un processo a priorità più bassa
- Più complicato se il processo è prelazionato da un altro ancora
- Priority-inheritance protocol



Inversione di Priorità

- ❑ L'inversione di priorità può avere effetti a catena causando un fallimento sistemico
- ❑ Il caso della missione **Mars Pathfinder**
 - ❑ Prima missione ad aver trasportato un rover, Sojourner, sul pianeta.
 - ❑ Lancio: 4 dicembre 1996
 - ❑ Arrivo: 4 luglio 1997 (7 mesi dopo), operatività 92 sol (95 giorni)
 - ❑ Lander operava come stazione meteorologica
 - ❑ Rover usato per analizzare il suolo e le rocce del sito di atterraggio ed effettuare esperimenti sulla superficie (100 metri di esplorazione)
 - ❑ https://space.skyrocket.de/doc_sdat/mars_pathfinder.htm



Inversione di Priorità

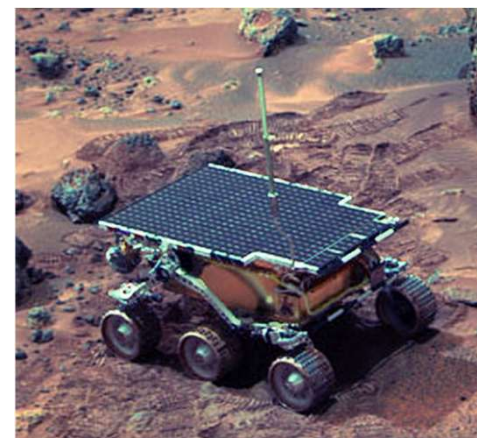
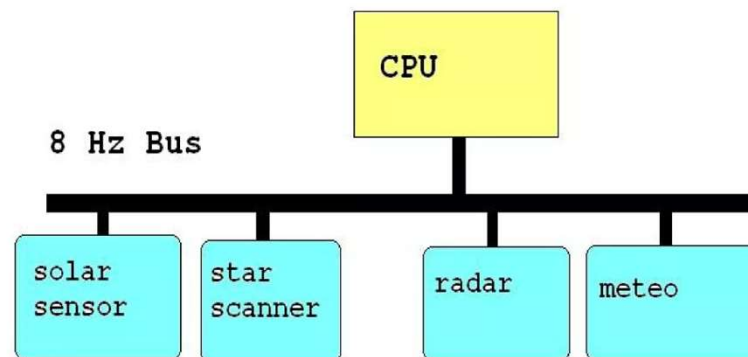
- ❑ L'inversione di priorità può avere effetti a catena causando un fallimento sistemico
- ❑ Il caso del Mars Pathfinder
 - ❑ Dopo i primi test il software di Pathfinder fa continui “reset”
 - ❑ Inizializzato hw, sw, comunicazione
- La raccolta dati dal bus era affidata a due task:
 - Il gestore delle transizioni del bus (`bc_sched`)
 - Il responsabile di acquisizione dati (`bc_dist`)
- Una sequenza di operazioni corrette richiedeva che i due task si alternassero in esecuzione, durante ogni ciclo di 8Hz
 - Quando un task iniziava, per prima cosa controllava che l'altro task avesse finito; altrimenti, generava un **reset del sistema**.



Inversione di Priorità

- L'inversione di priorità può avere effetti a catena causando un fallimento sistemico
- Il caso del Mars Pathfinder
 - Dopo i primi test il software di Pathfinder fa continui “reset”
 - Inizializzato hw, sw, comunicazione

- La raccolta dati dal bus era affidata a due task:
 - Il gestore delle transizioni del bus (`bc_sched`)
 - Il responsabile di acquisizione dati (`bc_dist`)
- Una sequenza di operazioni corrette richiedeva che i due task si alternassero in esecuzione, durante ogni ciclo di 8Hz
 - Quando un task iniziava, per prima cosa controllava che l'altro task avesse finito; altrimenti, generava un **reset del sistema**.



•Attivazione bus

- `bc_dist`: distribuisce i dati e passa il controllo a `bc_sched`
- `bc_sched`: bus scheduler prepara lo schedulere per il prossimo ciclo basandosi su priorità e dati preparati da `bc_dist` (meteo comunica direttamente a `bc_sched`)

Inversione di Priorità

- ❑ L'inversione di priorità può avere effetti a catena causando un fallimento sistemico
- ❑ Il caso del Mars Pathfinder
 - ❑ Dopo i primi test il software di Sojourner fa continui "reset"
 - ❑ Inizializzato hw, sw, comunicazione

Priorità:

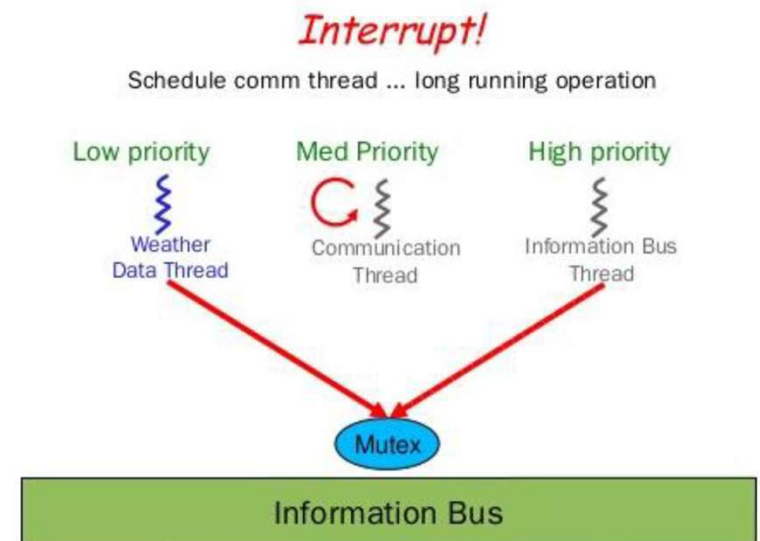
1 bc_sched, 3 bc_dist, 4 operazioni stazione, 5 esperimenti scientifici

Il problema riguardava tre task:

- bc_dist (priorità 3),
- Un qualsiasi task relativo alle operazioni della stazione spaziale (priorità 4), e
- Il task della stazione meteo (priorità 5)

Il task della stazione meteo riceveva i dati da bc_sched, per fare questo, doveva **prendere in uso esclusivo (lock) una risorsa, cioè il bus**.

- La sua esecuzione veniva sospesa dal task con priorità 4, prima che rilasciasse il lock,
- Successivamente, bc_dist non riusciva ad acquisire quella risorsa, e forzava il reset.

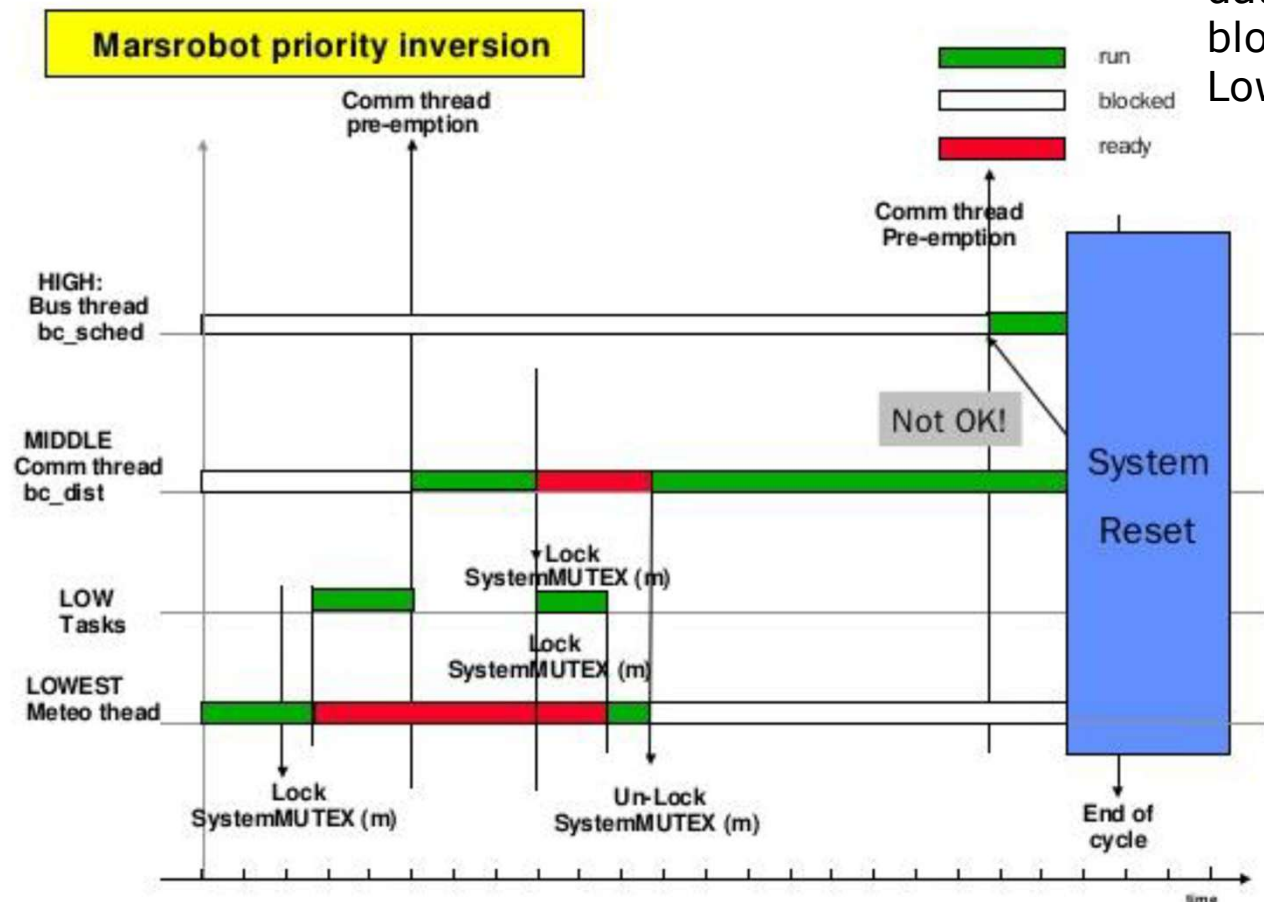


Inversione di Priorità

Il caso del Mars Pathfinder

- Dopo i primi test il software di Sojourner fa continui “reset”
- Inizializzato hw, sw, comunicazione

Stazione meteo riceve dati da bc_sched, blocca il bus
Low task prelaiona



<https://www.slideshare.net/gabrielladodero/quando-i-computer-non-funzionano-su-marte>

Inversione di Priorità

- Il caso del Mars Pathfinder
 - Dopo i primi test il software di Pathfinder fa continui “reset”
 - Inizializzato hw, sw, comunicazione

 - SO su Pathfinder era VxWorks (real-time)
 - Variabile globale per priority inheritance su tutti i semafori
 - La variabile settata su Pathfinder e il problema è stato risolto

<https://www.slideshare.net/gabrielladodero/quando-i-computer-non-funzionano-su-marte>

Livelli di Contesa

- ❑ Strumenti per il problema della sezione critica e per sincronizzare l'attività dei processi possono essere valutati a seconda del livello di contesa.
- ❑ Alcuni strumenti funzionano meglio con un certo livello di contesa
- ❑ Strumenti basati su istruzioni CAS per avere **lock-free**
 - ❑ Approccio ottimistico (prima test poi verifica collisioni)
 - ❑ Pessimistico (prima lock poi test)

```
typedef struct node {  
    value_t data;  
    struct node *next;  
} Node;  
  
Node *top; // top of stack
```

Treiber 86

```
void push(value_t item) {  
    Node *old_node;  
    Node *new_node;  
  
    new_node = malloc(sizeof(Node));  
    new_node->data = item;  
  
    do {  
        old_node = top;  
        new_node->next = old_node;  
    }  
    while (compare_and_swap(top, old_node, new_node) != old_node);  
}
```

Livelli di Contesa

- ❑ Strumenti per il problema della sezione critica e per sincronizzare l'attività dei processi possono essere valutati a seconda del livello di contesa.
- ❑ Alcuni strumenti funzionano meglio con un certo livello di contesa
- ❑ Strumenti basati su istruzioni CAS per avere **lock-free**
 - ❑ Approccio ottimistico (prima test poi verifica collisioni)
 - ❑ Pessimistico (prima lock poi test)

```
typedef struct node {  
    value_t data;  
    struct node *next;  
} Node;  
  
Node *top; // top of stack
```

Treiber 86

```
value_t pop() {  
    Node *old_node;  
    Node *new_node;  
  
    do {  
        old_node = top;  
        if (old_node == NULL)  
            return NULL;  
        new_node = old_node->next;  
    }  
    while (compare_and_swap(top, old_node, new_node) != old_node);  
  
    return old_node->data;  
}
```

Livelli di Contesa

- ❑ Strumenti per il problema della sezione critica e per sincronizzare l'attività dei processi possono essere valutati a seconda del livello di contesa.
- ❑ Alcuni strumenti funzionano meglio con un certo livello di contesa
- ❑ Strumenti basati su istruzioni CAS per avere lock-free
 - ❑ Approccio ottimistico (prima testi poi verifichi collisioni)
 - ❑ Pessimistico (prima lock poi test)
- ❑ Linee guida per distinguere le prestazioni della sincronizzazione al variare del livello di contesa:
 - ❑ **Nessuna contesa.** Sebbene entrambe le opzioni siano generalmente veloci, la protezione ottimistica sarà leggermente più veloce della sincronizzazione tradizionale.
 - ❑ **Contesa moderata.** La protezione ottimistica sarà più veloce e in alcuni casi molto più veloce rispetto alla sincronizzazione tradizionale.
 - ❑ **Alta contesa.** Con carichi molto elevati, la sincronizzazione tradizionale sarà in definitiva più veloce della sincronizzazione ottimistica.

Valutazione

- La scelta del meccanismo di sincronizzazione influenza le performance.
 - Interi atomici più leggeri dei lock e appropriati per update singoli di variabili (es. contatori)
 - SpinLock appropriati per multiprocessori se il lock è di breve durata
 - Mutex lock solitamente più leggeri dei semafori contatori e binari
 - Per il controllo di accesso a risorse multiple più appropriati i semafori contatori
 - Solitamente strutture di più alto livello come i monitor hanno maggiore overhead